

รายงานโปรเจกต์: Network Programmability Demo

Repository: [LynxelSins/NetworkProgrammability](#)
ชื่อเต็ม: Network Programmability Demo (mock RESTCONF, single Fedora box)
Presentation: [Canva Slide](#)

1. ภาพรวมโปรเจกต์ (Overview)

โปรเจกต์นี้เป็น การสาธิตเปรียบเทียบการจัดการเครือข่ายแบบดั้งเดิม (Traditional CLI) กับ การจัดการเครือข่ายแบบโปรแกรมได้ (Network Programmability ผ่าน RESTCONF/YANG) โดยจำลองทั้งหมดไว้บนเครื่อง Fedora เครื่องเดียว โดยไม่ต้องใช้ IOS หรือ GNS3 จริง

แนวคิดหลักคือการสร้าง "router ปลอม" ที่มีสถานะ (state) ชุดเดียว แล้วให้เครื่องมือ 2 แบบ — CLI สไตล์ Cisco และไคลเอนต์ RESTCONF — เข้าถึง/แก้ไข state เดียวกันผ่าน HTTP จริง เพื่อพิสูจน์ว่าเมื่อสั่งคำสั่งจากฝั่งหนึ่ง (เช่น `no shutdown` ผ่าน CLI) อีกฝั่งหนึ่ง (เช่น `demo_client.py get`) จะเห็นผลลัพธ์ตรงกันทันที ไม่ใช่ mock คนละชุดกัน

โปรเจกต์ถูกออกแบบมาเป็น **สคริปต์ประกอบการนำเสนอ (demo script)** แบ่งเป็น Act ต่าง ๆ (Act I, Act VI, Act VII) ซึ่งสอดคล้องกับสไลด์นำเสนอ

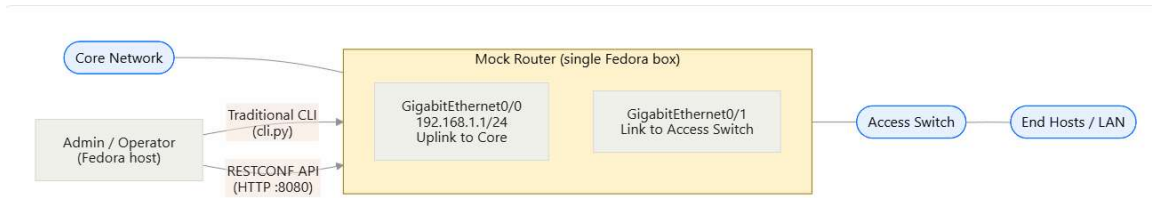
2. โครงสร้างไฟล์ในโปรเจกต์

ไฟล์	บทบาท
<code>server.py</code>	Mock RESTCONF/YANG server (FastAPI) — เป็น management interface แบบโปรแกรมได้
<code>router_core.py</code>	Core state engine ของ router จำลอง — single source of truth
<code>cli.py</code>	จำลอง Cisco-style CLI แบบดั้งเดิม (traditional box-by-box management)
<code>demo_client.py</code>	สคริปต์ Python ที่สาธิตการคุยกับ router ผ่าน RESTCONF (ใช้ใน Act VI/VII)
<code>gui.py</code>	หน้าจอ Visualizer แบบ interactive (Pygame) แสดง CLI + RESTCONF + LED สถานะ router + ปุ่มควบคุม
<code>router_state.json</code>	ไฟล์เก็บ state ปัจจุบันของ interface ต่าง ๆ (persist state)

ไฟล์	บทบาท
requirements.txt	รายการ dependency ของโปรเจกต์
README.md	คำอธิบายและวิธีติดตั้ง/รันโปรเจกต์

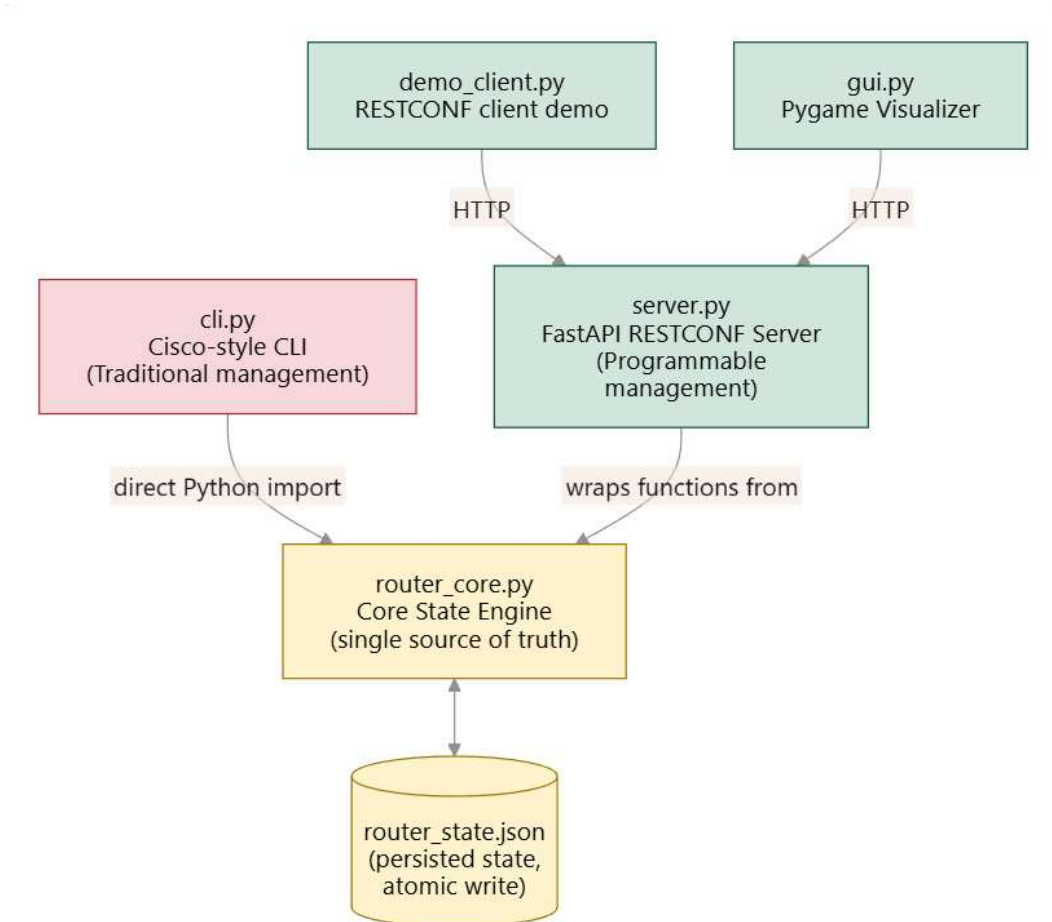
3. Network Topology (จำลอง)

Router ที่จำลองในโปรเจกต์มี 2 interface ตามค่าเริ่มต้นใน `router_core.py` ได้แก่ `GigabitEthernet0/0` (เชื่อมไปยัง Core) และ `GigabitEthernet0/1` (เชื่อมไปยัง Access Switch) โดยผู้ดูแลระบบเข้าถึง/จัดการ router ตัวนี้ได้ 2 ทาง คือผ่าน CLI โดยตรง หรือผ่าน RESTCONF API:



4. สถาปัตยกรรมระบบ (Architecture)

โครงสร้างเป็นแบบ **shared-state** ผ่าน **HTTP** โดยมี `router_core.py` เป็นแกนกลาง:



ประเด็นสำคัญทางสถาปัตยกรรม:

- `cli.py` เข้าถึง state โดยตรงผ่าน Python function call ไปยัง `router_core.py` (import โดยตรง) — จำลองการจัดการแบบ box-by-box แบบเดิม ที่ไม่มี API layer
- `server.py` ห่อ (wrap) ฟังก์ชันเดียวกันจาก `router_core.py` ด้วย FastAPI endpoints ตามมาตรฐาน RESTCONF (`ietf-interfaces` YANG model) — เป็นตัวแทนของการจัดการแบบโปรแกรมได้
- เพราะทั้งสองฝั่งอ้างอิง state เดียวกันผ่าน `router_state.json` การเปลี่ยนแปลงจากฝั่งหนึ่งจะสะท้อนที่อีกฝั่งทันที — นี่คือหัวใจของการสาธิต

5. รายละเอียดทางเทคนิค (Technical Details)

5.1 `router_core.py` — Core State Engine

- เก็บ state เป็น dict ของ `interface` list โดยแต่ละ interface มีฟิลด์ตามแนวทาง YANG model `ietf-interfaces` (เช่น `name`, `type`, `enabled`, `ipv4.address`, `oper-status`)
- ค่าเริ่มต้น (DEFAULT_STATE) มี 2 interface: `GigabitEthernet0/0` (Uplink to Core) และ `GigabitEthernet0/1` (Link to Access Switch)
- ฟังก์ชันหลัก:
 - `normalize_interface_name()` — แปลงชื่อย่อแบบ Cisco (เช่น `g0/0`, `gi0/0`) ให้เป็นชื่อเต็ม (`GigabitEthernet0/0`)
 - `_load_state()` / `_save_state()` — โหลด/บันทึก state จากไฟล์ JSON แบบ **atomic write** (เขียนลง `.tmp` แล้ว `replace()` เพื่อป้องกันไฟล์เสียหาย)
 - `get_all_interfaces()`, `get_interface()` — อ่านข้อมูล interface
 - `set_interface_enabled()`, `set_interface_ip()`, `update_interface()` — แก้ไขสถานะ พร้อมอัปเดต `oper-status` ให้สอดคล้องกับ `enabled` โดยอัตโนมัติ
 - `reset_state()` — คืนค่า state กลับเป็นค่าเริ่มต้น (ใช้สำหรับรีเซ็ตก่อน/หลังสาธิต)

5.2 `server.py` — Mock RESTCONF Server (FastAPI)

- ใช้ **FastAPI** เป็น web framework, ส่งข้อมูลด้วย content-type `application/yang-data+json` ตามมาตรฐาน RESTCONF
- Endpoints หลัก:
 - `GET /restconf/data/ietf-interfaces:interfaces` — ดึงข้อมูล interface ทั้งหมด
 - `GET /restconf/data/ietf-interfaces:interfaces/interface={name}` — ดึงข้อมูล interface เดียว (คืน 404 ถ้าไม่พบ)

- PATCH
/restconf/data/ietf-interfaces:interfaces/interface={name}
— แก้ไขค่าของ interface (เช่น enabled, ipv4)
- Endpoints เสริมสำหรับควบคุมการสาธิต ("Director's remote" สำหรับ Act VII):
 - POST /simulate/down / POST /simulate/up — จำลองอุปกรณ์/transport ล่ม (ทุก request ไปยัง /restconf/* จะดีเลย์ 1.5 วินาทีแล้วคืน 503 device unreachable)
 - POST /simulate/reset — ปิดโหมด chaos และรีเซ็ต state กลับค่าเริ่มต้น
- ใช้ **middleware** (chaos_middleware) เพื่อสร้างสถานการณ์ "connection failed" แบบเรียลไทม์ระหว่างการสาธิตสด

5.3 cli.py — Traditional Cisco-style CLI

- สร้างด้วย Python **cmd** module จำลอง CLI แบบ Cisco IOS ครบ 4 โหมด: **user** → **priv** → **config** → **config-if**
- คำสั่งที่รองรับ: **enable, configure terminal, interface <name>, shutdown, no shutdown, ip address <ip> <mask>, show ip interface brief, show running-config, show ip route, exit, end, quit**
- **สำคัญ:** ไฟล์นี้ห้าม import **requests** หรือคุยกับ RESTCONF endpoint โดยเด็ดขาด (ระบุไว้ใน docstring) — เพื่อคงความหมายของ "Traditional management" ที่เข้าถึง device โดยตรงไม่ผ่าน API ชั้นกลาง
- **show running-config** และ **show ip route** สร้างผลลัพธ์เลียนแบบ IOS output จริง (เช่น การคำนวณ subnet แบบ /24 จาก IP ที่ตั้งไว้)

5.4 demo_client.py

- สคริปต์ตัวอย่างฝั่งไคลเอนต์ที่คุยกับ **server.py** ผ่าน RESTCONF (HTTP) เพื่อแสดงบทบาท "โปรแกรมคุยกับ router" ในช่วง Act VI/VII ของการสาธิต (เช่นคำสั่ง **python demo_client.py get**)

5.5 gui.py

- หน้าจอ Visualizer เชิง interactive ด้วย Pygame ที่รวมมุมมอง CLI, RESTCONF, LED แสดงสถานะ router และปุ่มควบคุม ไว้ในที่เดียวเพื่อใช้ประกอบการนำเสนอสด

6. Data Model (ietf-interfaces)

แต่ละ interface ใน state มีโครงสร้างประมาณนี้:

JSON

```
{
  "name": "GigabitEthernet0/0",
  "description": "Uplink to Core",
  "type": "iana-if-type:ethernetCsmacd",
  "enabled": true,
  "ipv4": {
    "address": [
      {"ip": "192.168.1.1", "netmask": "255.255.255.0"}
    ]
  },
  "oper-status": "up"
}
```

7. การติดตั้งและใช้งาน (Setup & Usage)

Shell

```
# 1) ติดตั้ง (ครั้งเดียว)
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

เปิด 3 terminal (หรือ 3 tmux pane):

Terminal	คำสั่ง	บทบาท
1	<code>uvicorn server:app</code> <code>--host 127.0.0.1</code> <code>--port 8080</code>	รัน RESTCONF server (เปิด ทั้งไวด์ลอด demo)
2	<code>python cli.py</code>	CLI แบบดั้งเดิม (Act I)
3	<code>python demo_client.py</code> <code>get</code>	RESTCONF client (Act VI/VII)

ตัวอย่างการใช้งาน CLI:

None ▼

Router> enable



```
Router# configure terminal
Router(config)# interface g0/0
Router(config-if)# no shutdown
Router(config-if)# end
Router# show ip interface brief
```

8. จุดเด่นของการออกแบบ

1. **Single source of truth** — ทุกช่องทางการเข้าถึง (CLI, RESTCONF, GUI) อ้างอิง state เดียวกันผ่าน `router_core.py/router_state.json` ทำให้การสาธิต "แก้จาก CLI แล้วเห็นผลใน RESTCONF" เป็นของจริง ไม่ใช่การจำลองแยกส่วน
2. **Atomic file write** — ป้องกัน state ไฟล์เสียหายระหว่างการเขียนพร้อมกัน
3. **Chaos middleware** — จำลองสถานการณ์อุปกรณ์ล่ม/timeout ได้แบบควบคุมได้ผ่าน API (`/simulate/down`, `/simulate/up`) เหมาะสำหรับสาธิตสัดโดยไม่ต้องปิดเครื่องจริง
4. **แยกความรับผิดชอบชัดเจน (Separation of concerns)** — `cli.py` ห้ามยุ่งกับ HTTP/RESTCONF โดยตั้งใจ เพื่อให้เห็นความแตกต่างระหว่าง Traditional management กับ Network Programmability อย่างชัดเจนในเชิงสถาปัตยกรรม ไม่ใช่แค่ผิวหน้า

9. ข้อสังเกต/จุดที่อาจพัฒนาต่อ

- ปัจจุบันรองรับเฉพาะ IPv4 address เดียวต่อ interface (`ipv4.address[0]`) — ยังไม่รองรับหลาย address หรือ IPv6
- `router_state.json` ถูก commit ไว้ใน repo (มีอยู่ในรายการไฟล์) ซึ่งหมายความว่า state ปัจจุบันของ demo ถูกเก็บลง version control ด้วย อาจพิจารณาใส่ใน `.gitignore` หากไม่ต้องการ track การเปลี่ยนแปลง state ระหว่างการทดสอบ
- โฟลเดอร์ `__pycache__` ปรากฏใน repo — ควรเพิ่มใน `.gitignore`
- README ยังไม่มีคำอธิบายเพิ่มเติมในส่วน "เอกสารที่เกี่ยวข้อง" (มีหัวข้อไว้แต่ยังว่าง)